

Milestone 2 Report: Space, Transformation & Camera

Procedural Terrain Generator

For Milestone 2, the procedural terrain generator was enhanced to represent terrain within a fully spatial three-dimensional environment. Building on the procedural generation techniques developed in Milestone 1, the project introduces model transformations, a virtual camera and three-dimensional visualization to improve the representation and interaction of the terrain.

The terrain continues to be generated mathematically using layered sine and cosine functions combined with controlled random variation. However, instead of displaying the terrain as a two-dimensional height map, the system now renders it as a three dimensional surface, allowing users to observe the landscape from different viewing angles. These improvements demonstrate the principles of three-dimensional space, transformations and camera systems while maintaining the procedural approach to terrain generation.

Group A members

SCT 221-C004-0046/2024- Nasteha Ahmed

SCT221-C004-0581/2024- Lorene Wambui

SCT221-C004-0561/2024- Reyhana Abdi

SCT221-C004-0093/2024- Robert Bashir

SCT221-C004-0263/2024- Cecilia Wambui

1. Introduction

The objective of this milestone is to extend the procedural terrain generator into a fully spatial three-dimensional system. While Milestone 1 focused on generating terrain mathematically, Milestone 2 focuses on representing that terrain correctly in 3D space through transformations, camera positioning and projection.

2. Fully Spatial 3D System

The terrain is represented as a collection of vertices arranged in a two-dimensional grid, where each vertex stores three coordinates:

X – Horizontal position

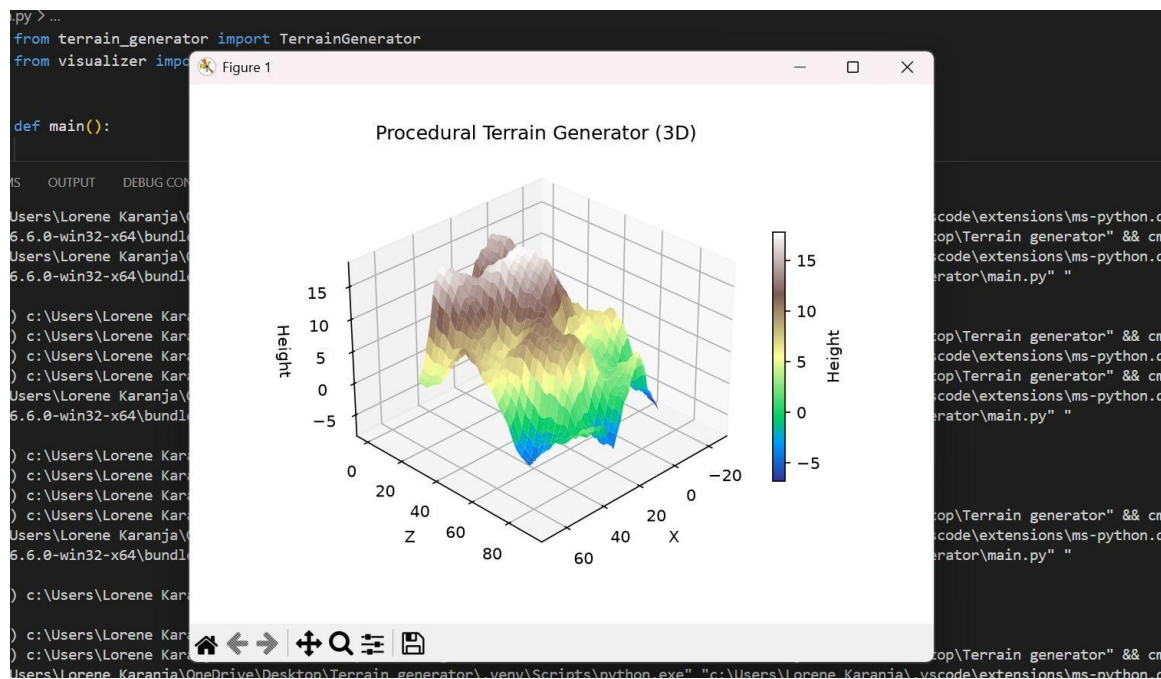
Y – Terrain elevation (height)

Z – Depth position

Each vertex is represented mathematically as:

$$V = (x, y, z)$$

Unlike the previous milestone, where the terrain was displayed as a height map, the vertices are now rendered as a continuous three-dimensional surface using Matplotlib's `plot surface ()` function. This allows the terrain to be viewed from multiple perspectives while preserving the spatial relationship between all vertices.



Source code

<https://github.com/Lokaalee/Procedural-Terrain-Generator>

3. Transformation Pipeline Implementation

Before rendering, each terrain vertex passes through a transformation pipeline consisting of model transformation and camera transformation

Model Transformation

The first stage applies scaling to increase the spacing between terrain vertices.

The scaling equations are: $x' = x \times 1.2$ $z' = z \times 1.2$

Scaling enlarges the terrain while maintaining its proportions. The terrain is then rotated around the Y-axis using the equations: $x' = x$

$\cos \theta - z \sin \theta$

$z' = x \sin \theta + z \cos \theta$

where $\theta = 20^\circ$

This rotation allows the terrain to be viewed from an angled perspective rather than directly from above.

Camera Model

A virtual camera is used to determine how the terrain is viewed.

The camera is positioned using:

where:

- Elevation (30°) controls the vertical viewing angle.
- Azimuth (45°) rotates the camera horizontally around the terrain.

4. Transformation Pipeline Representation

Terrain Generation



Vertex Grid



Scaling



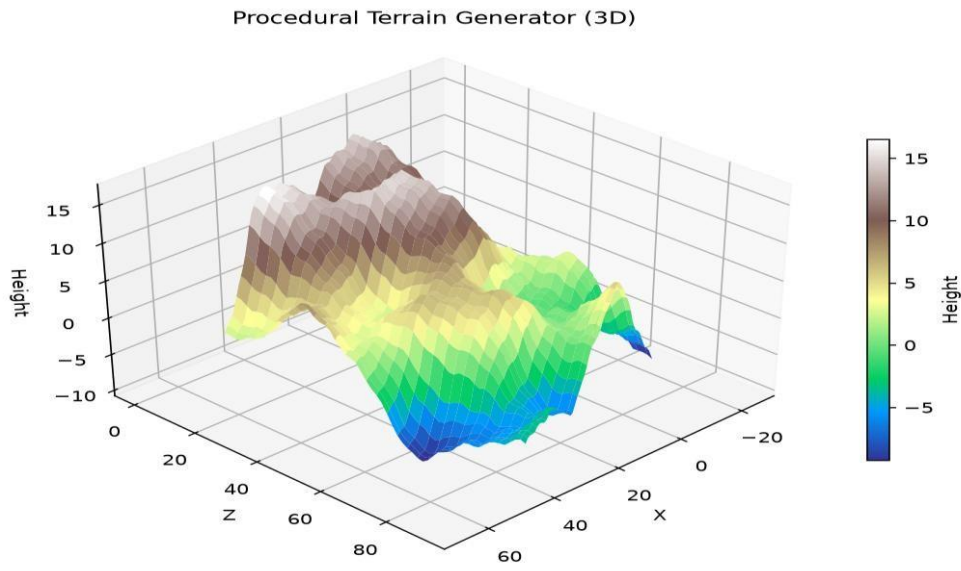
Rotation



Camera View



Perspective Projection



5. Numerical Stability Analysis

The terrain height is computed as

$$\text{height} = (1.2 \times \text{large}) + \text{medium} + \text{small} + \text{detail}$$

6. System and Hardware Constraint Awareness

The implementation was designed to operate on a standard personal computer using Python and Matplotlib.

The system relies on:

- CPU to perform mathematical calculations and generate terrain elevations.
- RAM to store the terrain grid and vertex data during execution.
- Graphics hardware through Matplotlib's rendering engine to display the three dimensional terrain.

Increasing the terrain resolution increases the number of vertices that must be generated and rendered, resulting in greater processing time and memory usage. A terrain size of 60×60 vertices was selected to balance rendering performance with visual quality.